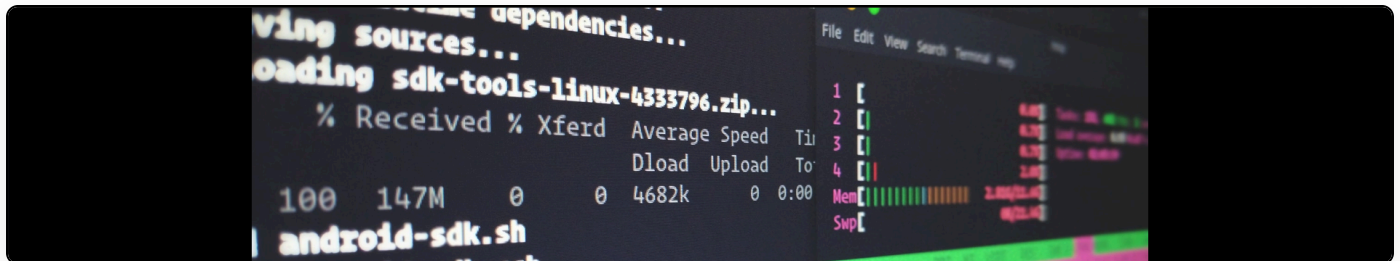


Bypassing firewalls using SSH Reverse Forwarding

Cite as: Martin Paul Eve, "Bypassing firewalls using SSH Reverse Forwarding", *eve.gd: Martin Paul Eve*, December 07, 2008, <https://doi.org/10.59348/8e1wk-2wf36>.



Sometimes you will find yourself on a machine that has no inbound connections allowed, which can make even the most basic task a complete pain. Never fear, if you can initiate an outbound connection to an SSH server of your choosing, it is no problem whatsoever to remap the ports by what is known as Reverse Forwarding.

The first step is to set up [key-based, passwordless authentication in SSH](#).

Secondly, we need to do a little bit of configuration to the SSH server, so become root:

```
su -
```

Then, edit `/etc/ssh/sshd_config` to include the line:

```
GatewayPorts yes
```

This enables reverse forwarding to listen on hosts other than localhost.

Next up, if your client is Windows you can add Tunnels under SSH -> Tunnels in PuTTY. A "remote" tunnel means that any connection coming in to the server will be forwarded to your client. A "dynamic" tunnel means that a SOCKS5 server will be setup locally on the client on the port you specify, allowing you to have an encrypted browsing session; very usefful for wireless hotspots.

The equivalent for Nix is:

```
ssh -R [host:]serverport:localhost:port -l server_user_name server.address
```

or for dynamic:

```
ssh -D port -l server_user_name server_address
```

Note that "localhost" in the above example will map to the client. So while it is possible to map to any server, localhost refers to the place where your client machine is: you will usually want to use localhost.

So now, any connection that comes to the mapped port on the server will be channeled down the SSH connection and the client software (PuTTY or SSH) will make a connection from the client, to the client, inside the firewall. Tada, firewall bypassed!